PROJECT REPORT

PERSONAL FINANCE ADVISOR BOT

Income Tracking, Smart Budgeting and Rule-Based AI Financial Insights for Indian Users

Submitted for

NASSCOM Virtual Internship Program – AI Specialist - BFSI

Submitted by

Pradnya Sandesh Tambade

Roll No: 2226 | Division: B | Branch: AIDS

Project Repository

https://github.com/pradnyatambade5-cpu/nasccom-11

Abstract

Personal Finance Advisor Bot is a full-stack web application that helps users in India record their income and expenses, set category-wise monthly budgets, track savings goals and receive plain-language financial guidance in Indian Rupees (₹). The application brings together authentication, income and expense management, budget monitoring, savings-goal tracking, four interactive analytical charts, a monthly report with CSV export, and a rule-based AI advisor with an interactive chat interface.

The advisor is rule-based and runs locally: it audits the user's recorded data using savings-rate and 50/30/20 proportions and overspending triggers, and returns categorized educational suggestions. It does not call an external AI service. The project is positioned as a financial-literacy and decision-support tool. It is not a registered financial adviser and does not replace professional advice, and its output depends entirely on the data the user enters.

1. Introduction

Managing personal money well depends on knowing how much comes in, where it goes, and whether spending matches plans. Many students and young professionals track this informally, if at all, and find it hard to notice overspending or to estimate how much they must save each month to reach a goal. Personal Finance Advisor Bot places these activities in a single dashboard and turns the recorded numbers into understandable guidance.

The project aligns with the AI in Banking, Financial Services and Insurance (BFSI) domain by combining web-based financial calculations with an algorithmic advisory layer designed for Indian users. It supports awareness and planning only.

2. Problem Statement

Users often lack a consolidated view of their income, category-wise spending, budget limits and savings progress. Without it, overspending goes unnoticed, savings goals stall, and monthly statements must be compiled by hand. This project addresses the gap with a secure multi-user web application that records transactions, compares spending against budgets, visualizes trends and produces structured advice and exportable monthly reports.

3. Objectives

- Provide secure registration and login with per-user data isolation.
- Record multiple income streams and categorized daily expenses with full create, read, update and delete (CRUD) support.
- Let users set category-wise monthly budgets and see clear status indicators (On Track, Near Limit, Budget Exceeded).
- Track savings goals with progress bars, remaining amounts and required monthly savings.
- Generate rule-based, categorized financial advice and answer common questions through an interactive chat bot.
- Visualize income, expenses, savings and budget performance through Chart.js charts.
- Provide filterable monthly reports with one-click CSV export and a print-ready layout.
- Promote financial literacy for Indian users through ₹-formatted values and plain-language explanations.

4. Scope and Key Features

4.1 Authentication and Data Privacy

Users register and log in with passwords stored as Werkzeug hashes, and sessions are managed with Flask-Login. Unauthenticated visitors are redirected to the login page with a prompt. Each user's financial records are linked to their own account so that data is kept separate between users. A one-click demo login (demo@example.com) is provided and is pre-loaded with realistic sample data for demonstration.

4.2 Dashboard Overview

The dashboard summarizes the current month: monthly income, total expenses and related indicators, along with four charts: Income vs Expenses over six months (bar), Expense by Category (doughnut), Monthly Savings (line/area) and Budget vs Actual (grouped bar). Chart data is delivered as JSON from a dedicated endpoint, which the front end renders with Chart.js.

4.3 Income Management

Users can add, view, edit and delete income entries across six sources: Salary, Freelance, Allowance, Scholarship, Business and Other. Monthly totals are calculated in real time.

4.4 Expense Tracking

Expenses are classified into eleven categories: Food, Rent, Transport, Education, Healthcare, Shopping, Entertainment, Utilities, Bills, Travel and Other. The page shows category breakdown pills and a chronological history of entries.

4.5 Category Budgets

Users define a monthly spending limit for each category. Progress bars compare actual spending against the limit and show a status of On Track, Near Limit or Budget Exceeded.

4.6 Savings Goals

Users set goals such as a gadget, an emergency fund or a vacation, each with a target amount, amount saved so far and a deadline. The page shows a progress bar, the remaining amount and the monthly saving required to meet the deadline. A quick-deposit action adds money to a goal in one click.

4.7 Rule-Based AI Advisor and Chat Bot

The advisor audits the user's recorded data and produces structured advice in five sections: Summary, Spending Analysis, Overspending Alerts, Savings Suggestions and Budget Suggestions. It evaluates 50/30/20 proportions, the savings rate and overspending triggers. An interactive chat bot offers suggested prompts such as “Where am I overspending?”, “How can I save more?”, “Analyze my spending”, “Give me a budget suggestion” and “Am I on track for my savings goals?”. Every advisor page carries a disclaimer that the insights are for informational purposes only.

4.8 Monthly Reports and CSV Export

The reports page presents a monthly financial statement filterable by month and year. Users can download the statement as a CSV file or print it using a print-ready layout.

4.9 Profile

A profile page displays the logged-in user's account details.

5. Technology and Architecture

The technology summary below is based on the project's README, requirements file and source code.

Layer	Technology
Backend	Python, Flask 3.1.0 (application factory pattern, blueprints for modular routes)
ORM / Database	Flask-SQLAlchemy 3.1.1 with SQLite; tables are created automatically at start-up
Authentication	Flask-Login 0.6.3 for sessions; Werkzeug 3.1.3 for password hashing
Configuration	python-dotenv 1.0.1; settings read from environment or a .env file
Validation	email-validator 2.2.0
Frontend	HTML5, vanilla CSS3 (custom FinTech design system), vanilla JavaScript (ES6+), Jinja2 templates
Charts and icons	Chart.js and RemixIcon
Testing	Python unittest with the Flask test client

5.1 Application Structure

The app is built with Flask's application factory (create_app). At start-up it loads configuration, initializes the database and login manager, registers all blueprints, registers two Jinja filters for rupee formatting (inr with two decimals and inr_int without decimals), sets custom 404 and 500 error pages, creates the tables and seeds the demo user.

The data model holds five entities: User, Income, Expense, Budget and SavingsGoal. Every record except the user itself carries a user identifier, which is what keeps each user's data separate.

5.2 Modules and Routes

Route	Purpose
/ and /login	Landing page and login (with one-click demo login)
/dashboard/ and /dashboard/api/charts	Dashboard overview and JSON data for the four charts
/income/ and /income/add	List and add income records (edit and delete also supported)
/expenses/ and /expenses/add	List and add expense records (edit and delete also supported)
/budget/	Category budgets with progress and status
/savings/	Savings goals, progress and quick deposits
/advisor/ and /advisor/chat	Rule-based advice page and chat endpoint
/reports/ and /reports/export/csv	Monthly statement and CSV download
/profile/	User profile page

5.3 High-Level Workflow

- The user registers or logs in (or uses the demo account).
- The user records income and expenses, sets budgets and creates savings goals.
- The backend validates the inputs and stores them in SQLite, linked to that user.
- The dashboard and the charts endpoint aggregate the user's data for the selected period.

- The advisor evaluates the data against its rules and returns categorized advice or a chat reply.
- The user reviews the monthly report and exports it as CSV if needed.

6. Financial Metrics and Logic

The advisor and dashboard rely on a small set of standard personal-finance measures. The worked examples use the pre-loaded demo user's current-month data (income ₹50,000; expenses ₹32,000). These are sample values created by the seed routine, not real financial records.

6.1 Monthly Savings and Savings Rate

Monthly savings equal total income minus total expenses, and the savings rate is savings divided by income, multiplied by 100. For the demo data, ₹50,000 − ₹32,000 = ₹18,000, giving a savings rate of 36%.

6.2 The 50/30/20 Guideline

The 50/30/20 rule suggests that about 50% of income goes to needs, 30% to wants and 20% to savings. The advisor compares the user's actual proportions with this guideline. It is a general rule of thumb and may not suit every household.

6.3 Budget Utilization and Overspending Triggers

Budget utilization is the amount spent in a category divided by that category's limit. Spending above the limit is flagged as Budget Exceeded. In the demo data, Food is ₹5,500 against a ₹5,000 budget and Entertainment is ₹2,500 against ₹2,000, so both are flagged, while the other five categories are within their limits.

6.4 Savings Goal Progress

Goal progress is the amount saved divided by the target. The required monthly saving is the remaining amount divided by the months left until the deadline. In the demo data, the New Laptop goal stands at ₹32,000 of ₹70,000 (about 46%) with ₹38,000 remaining, and the Emergency Fund goal at ₹45,000 of ₹100,000 (45%) with ₹55,000 remaining.

6.5 Demo Data Summary

Category	Spent (₹)	Budget (₹)	Status
Rent	12,000	12,000	At limit
Food	5,500	5,000	Exceeded
Transport	3,000	3,000	At limit
Education	4,000	4,000	At limit
Entertainment	2,500	2,000	Exceeded
Shopping	3,000	3,500	Within budget
Utilities	2,000	2,500	Within budget

8. Security, Privacy and Responsible AI

8.1 Implemented

- Passwords are stored as Werkzeug hashes, never in plain text.

- Flask-Login protects the application's pages and redirects unauthenticated users to the login screen.
- Financial records are tied to a user identifier so that users see only their own data.
- The secret key and database location can be set through environment variables or a .env file, and the .env file and database are excluded from version control by .gitignore.
- The advisor runs locally, so no personal financial data is sent to a third-party AI provider.
- A disclaimer on the advisor page states that insights are informational only.

8.2 Recommended Before Any Real-World Use

- Replace the built-in fallback secret key by setting a strong SECRET_KEY in the environment, and run with debug mode turned off.
- Remove or disable the seeded demo account and its published credentials in a production deployment.
- Use HTTPS, add CSRF protection to all forms and validate every numeric input on the server (non-negative amounts, valid dates, sensible limits).
- Add rate limiting on login and a data-deletion option, together with a privacy notice.
- Present advisor output as educational guidance. Thresholds such as 50/30/20 are indicative and vary by individual circumstances.

9. Testing and Evaluation

The repository includes an automated suite of 10 tests in test_app.py, written with Python's unittest and the Flask test client. Each test logs in with the demo account and checks that pages respond successfully and contain the expected content. This report does not claim that the tests were independently executed during its preparation.

Test	What it checks
test_landing_page	Landing page loads and shows its headline and feature text
test_demo_login_flow	Demo login succeeds and the dashboard shows income and expense cards
test_dashboard_charts_api	Chart endpoint returns JSON with the expected label and data keys
test_income_crud	Income page loads and a new income entry can be added and appears
test_expenses_crud	Expense page loads and a new expense can be added and appears
test_budget_management	Budget page loads with category budgets listed
test_savings_goals	Savings page loads and shows the seeded goal
test_ai_advisor_page_and_chat	Advisor page shows the disclaimer and the chat endpoint returns a reply
test_reports_and_csv	Report page loads and the CSV export returns the right content type
test_profile_page	Profile page loads and shows the user's name

A fuller validation plan would add unit tests for the financial formulas, tests that one user cannot access another user's records, input-validation tests for negative or malformed values, and a separate test database so that tests do not write into the development data.

10. Limitations

- All insights depend on the data the user enters; the application is not connected to banks, credit bureaus or payment systems.
- The advisor is rule-based, so it follows fixed rules and cannot reason about unusual situations or give personalized professional advice.
- Rules of thumb such as 50/30/20 may not suit every income level or household.
- SQLite is suitable for a single-server demonstration but not for large-scale multi-user production use.
- A production release would need privacy disclosures, secure secret management, backups, monitoring and deployment hardening.

11. Future Enhancements

- Add an LLM-based advisor with grounded prompts, alongside the existing rule engine, and evaluate its answers for factual accuracy and safety.
- Recurring transactions, due-date reminders and bill alerts.
- Import of bank statements (CSV or PDF) and automatic categorization of expenses.
- Multilingual support (for example Hindi and Marathi) for wider accessibility in India.
- PDF reports, additional chart types and year-on-year comparisons.
- Credit-health indicators such as debt-to-income ratio and credit utilization.
- Deployment on a cloud platform with a production database, HTTPS and audit logging.

12. Conclusion

Personal Finance Advisor Bot demonstrates a practical use of full-stack development and rule-based AI in the BFSI domain. By combining secure authentication, transaction tracking, budgets, savings goals, analytical charts, monthly reports and an advisory chat bot in one application, it makes everyday money management easier to understand for Indian users. The next stage should focus on hardening security settings, strengthening input validation and testing, and extending the advisor with evaluated AI capabilities.

References

- Project repository: https://github.com/pradnyatambade5-cpu/nasccom-11
- Flask documentation: https://flask.palletsprojects.com/
- Flask-SQLAlchemy documentation: https://flask-sqlalchemy.palletsprojects.com/
- Flask-Login documentation: https://flask-login.readthedocs.io/
- Werkzeug documentation: https://werkzeug.palletsprojects.com/
- Chart.js documentation: https://www.chartjs.org/docs/
- SQLite documentation: https://www.sqlite.org/docs.html